

NLP Project Repository Documentation

Project Title: Sentiment Analysis of Customer Reviews Using Deep Neural Networks

Course Assignment: Research, Model Implementation, and GitHub Repository Documentation

1. Project Overview

This project develops a Natural Language Processing (NLP) system for binary sentiment analysis using the IMDb Movie Reviews dataset. The objective is to automatically classify reviews as positive or negative using deep neural networks. The project demonstrates the complete machine learning lifecycle including literature review, dataset preparation, model development, training, evaluation, and reproducible deployment through a well-organized GitHub repository.

2. Research and Selection of Methods

The project begins with defining the sentiment classification objective. Recent literature on sentiment analysis, deep learning, word embeddings, LSTM networks, CNNs, and transformer-based models such as BERT was reviewed. A deep neural network with transfer learning concepts was selected because it provides an effective balance between accuracy, computational efficiency, and implementation complexity for the IMDb dataset. Preliminary experiments compare preprocessing techniques, vocabulary size, sequence length, and hyperparameters before final training.

3. Dataset Description

The IMDb Movie Reviews dataset contains 50,000 labeled reviews evenly divided into positive and negative classes. Reviews are split into 25,000 training and 25,000 testing samples. Text preprocessing includes lowercasing, punctuation removal, tokenization, padding, and vocabulary generation. The balanced nature of the dataset makes it suitable for evaluating binary sentiment classifiers.

4. Model Implementation

Python together with TensorFlow/Keras, NumPy, Pandas, NLTK, and Scikit-learn forms the implementation environment. The neural network consists of an Embedding layer, GlobalAveragePooling1D layer, Dense hidden layers with Dropout regularization, and a Sigmoid output layer. Binary Cross-Entropy is used as the loss function while Adam serves as the optimizer. Early stopping and model checkpointing are applied to reduce overfitting.

5. Training and Evaluation

The model is trained using mini-batches and validated throughout training. Performance is evaluated using Accuracy, Precision, Recall, F1-score, ROC-AUC, confusion matrix, classification report, and learning curves. Error analysis identifies common misclassified reviews and guides further improvements.

6. GitHub Repository

The repository follows a modular structure with folders for source code, notebooks, datasets, trained models, results, and documentation. Git is used for version control through feature branches

and pull requests. README documentation explains installation, dependencies, project structure, execution steps, and model design decisions.

7. Repository Structure

Root files include README.md, requirements.txt, and .gitignore. Subdirectories include data/, notebooks/, src/, models/, results/, and docs/. Documentation includes literature review, methodology, experimental findings, and final report. The repository is organized to maximize readability, reproducibility, and collaboration.

8. Expected Deliverables

Deliverables include cleaned datasets, preprocessing scripts, training scripts, evaluation scripts, trained model weights, performance graphs, confusion matrices, GitHub documentation, and the final project report.

9. Conclusion

This project demonstrates a complete NLP workflow from research through implementation and evaluation. The GitHub repository provides a reproducible environment that documents every phase of development, enabling reviewers to understand, reproduce, and extend the work with minimal effort.

References

Brownlee, J. (2020). Deep Learning for Natural Language Processing.
Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning.
Maas, A. L., et al. (2011). Learning Word Vectors for Sentiment Analysis.
Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach.